

DB JS

```
JS db.js ×
backend > src > config > JS db.js > ...
1  import mysql from "mysql2/promise";
2  import "dotenv/config";
3
4  export const pool = mysql.createPool({
5    host: process.env.DB_HOST,
6    port: Number(process.env.DB_PORT || 3306),
7    user: process.env.DB_USER,
8    password: process.env.DB_PASSWORD,
9    database: process.env.DB_NAME,
10
11    // define se quem chegar depois espera;
12    waitForConnections: true,
13    // define quantas conexões trabalham ao mesmo tempo;
14    connectionLimit: 10,
15    // queueLimit define quantos podem ficar nessa espera.
16    queueLimit: 0
17
18    // é basicamente: "Trabalhe com até 10 conexões;
19    // se lotar, espere; e não imponha um limite de tamanho à fila."
20
21    //0 queueLimit: 0 não significa "sem fila"; significa fila sem limite configurado.
22  });
```

AUTHCONTROLLER JS

JS authController.js X

backend > src > controllers > JS authController.js > cadastrar

```
1 import bcrypt from "bcryptjs";
2 import jwt from "jsonwebtoken";
3 import { pool } from "../config/db.js";
4
```

```
5 export async function cadastrar(req, res) {
6   try {
7     const { usuario, senha } = req.body;
8
9     if (!usuario || !senha) {
10       return res.status(400).json({ mensagem: "Usuário e senha são obrigatórios" });
11     }
12
13     if (senha.length < 4) {
14       return res.status(400).json({ mensagem: "A senha deve ter pelo menos 4 caracteres" });
15     }
16
17     const [existentes] = await pool.query(
18       "SELECT id FROM usuarios WHERE usuario = ? LIMIT 1",
19       [usuario]
20     );
21
22     if (existentes.length > 0) {
23       return res.status(409).json({ mensagem: "Usuário já cadastrado" });
24     }
25
26     const senhaHash = await bcrypt.hash(senha, 10);
27
28     const [resultado] = await pool.query(
29       "INSERT INTO usuarios (usuario, senha) VALUES (?, ?)",
30       [usuario, senhaHash]
31     );
32
33     return res.status(201).json({
34       mensagem: "Usuário cadastrado com sucesso",
35       usuario: { id: resultado.insertId, usuario }
36     });
37
38   } catch (erro) {
39     console.error(erro);
40     return res.status(500).json({ mensagem: "Erro ao cadastrar usuário" });
41   }
42 }
```

CRIAR A ROTA E O SERVER PARA TESTAR

```

43
44
45 export async function login(req, res) {
46   try {
47     const { usuario, senha } = req.body;
48
49     if (!usuario || !senha) {
50       return res.status(400).json({ mensagem: "Usuário e senha são obrigatórios" });
51     }
52
53     const [usuarios] = await pool.query(
54       "SELECT id, usuario, senha FROM usuarios WHERE usuario = ? LIMIT 1",
55       [usuario]
56     );
57
58     if (usuarios.length === 0) {
59       return res.status(401).json({ mensagem: "Usuário ou senha inválidos" });
60     }
61
62     const encontrado = usuarios[0];
63     const senhaValida = await bcrypt.compare(senha, encontrado.senha);
64
65     if (!senhaValida) {
66       return res.status(401).json({ mensagem: "Usuário ou senha inválidos" });
67     }
68
69     const token = jwt.sign(
70       { id: encontrado.id, usuario: encontrado.usuario },
71       process.env.JWT_SECRET,
72       { expiresIn: "2h" }
73     );
74
75     return res.json({
76       mensagem: "Login realizado com sucesso",
77       usuario: { id: encontrado.id, usuario: encontrado.usuario },
78       token
79     });
80
81   } catch (erro) {
82     console.error(erro);
83     return res.status(500).json({ mensagem: "Erro no login" });
84   }
85 }

```

{ expiresIn: "30s" } o token vence em 30 segundos
CRIAR A ROTA E O SERVER PARA TESTAR

```
86
87   export async function me(req, res) {
88     return res.json({ usuario: req.usuario });
89   }
90
91   export async function listarUsuarios(req, res) {
92     try {
93       const [usuarios] = await pool.query(
94         "SELECT id, usuario, criado_em FROM usuarios ORDER BY id"
95       );
96       return res.json(usuarios);
97     } catch (erro) {
98       console.error(erro);
99       return res.status(500).json({ mensagem: "Erro ao listar usuários" });
100    }
101  }
102
```

CRIAR A ROTA E O SERVER PARA TESTAR SOMENTE **listarUsuarios ()**

ROUTES JS

```
JS authRoutes.js X
backend > src > routes > JS authRoutes.js > ...
1  import { Router } from "express";
2  import {
3    cadastrar,
4    login,
5    me,
6    listarUsuarios
7  } from "../controllers/authController.js";
8  import { autenticar } from "../middlewares/authMiddleware.js";
9
10 const router = Router();
11
12 router.post("/usuarios", cadastrar);
13 router.post("/login", login);
14 router.get("/auth/me", autenticar, me);
15 router.get("/usuarios", autenticar, listarUsuarios);
16
17 export default router;
18
```

SERVER JS

```
JS server.js X
backend > src > JS server.js > ...
1  import express from "express";
2  import cors from "cors";
3  import "dotenv/config";
4  import authRoutes from "../routes/authRoutes.js";
5  import { pool } from "../config/db.js";
6
7  const app = express();
8  const PORT = process.env.PORT || 3000;
9
10 app.use(cors({ origin: "http://localhost:5173" }));
11 app.use(express.json());
12
13 app.get("/", (req, res) => {
14   res.json({ mensagem: "API de autenticação funcionando" });
15 });
16
17 app.use(authRoutes);
18
19 async function iniciar() {
20   try {
21
22     const connection = await pool.getConnection();
23     console.log("MySQL conectado com sucesso");
24     connection.release();
25
26
27     app.listen(PORT, () => {
28       console.log(`Servidor rodando em http://localhost:${PORT}`);
29     });
30
31   } catch (erro) {
32     console.error("Erro ao conectar no MySQL:", erro.message);
33     process.exit(1);
34   }
35 }
36
37 iniciar();
```

AUTH MIDDLEWARE JS

```
JS authMiddleware.js X
backend > src > middlewares > JS authMiddleware.js > ...
1  import jwt from "jsonwebtoken";
2
3  export function autenticar(req, res, next) {
4    const authHeader = req.headers.authorization;
5
6    if (!authHeader?.startsWith("Bearer ")) {
7      return res.status(401).json({ mensagem: "Token não informado" });
8    }
9
10   const token = authHeader.split(" ")[1];
11
12
13   try {
14
15     const payload = jwt.verify(token, process.env.JWT_SECRET);
16     req.usuario = payload;
17     next();
18   } catch {
19     return res.status(401).json({ mensagem: "Token inválido ou expirado" });
20   }
21 }
22
23 }
```

TC New Request X JS authController.js X

GET http://localhost:3000/usuarios Send

Query Headers 2 Auth 1 Body 1 Tests Pre Run

None Basic Bearer OAuth 2 NTLM AWS

Bearer Token

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im1hcmNvcylzImIhdCI6MTc4OTI0MTg1MSwiZXhwIjoxNzg5MjQxODg5fQ.dujz15nl1PEQN7_qCwNOKyTeFuf7QbCe44XtVZNZgV8
```

Status: 200 OK Size: 68 Bytes Time: 8 ms Response

```
1  [
2    {
3      "id": 1,
4      "usuario": "marcos",
5      "criado_em": "2026-09-12T19:34:08.000Z"
6    }
7  ]
```

TESTAR A ROTA /auth/me antes de começar o frontend